



INDIA.RUNS

Build what next India runs on

Team Name : sourabh-sonker

Team Leader Name : Sourabh Sonker

Problem Statement : Track 1: The Data & AI Challenge

Solution Overview

● What is your proposed solution?

- I designed and built a high-performance, deterministic Python-native candidate evaluation and ranking pipeline.
- The system processes candidate JSON structures using a stream-based parser, calculates individual candidate scores across four customized feature dimensions, and outputs a strictly validated candidate list.
- Includes an explicit, automated "Honeypot Shield" that detects and eliminates corrupted, logically impossible candidate records.

● What differentiates your approach from traditional candidate matching systems?

- **Defeating Keyword Stuffing:** Traditional systems blindly count skill matches. My engine weights each skill by a composite Trust Score derived from proficiency tiers, usage duration, and verified platform endorsements.
- **Strict Career Trajectory Tracking:** I implement an active-history analyzer that isolates candidates who have only worked at IT service consulting giants, while preserving high-quality candidates who transitioned into product-focused environments.
- **Reachable-First Optimization:** High-quality but inactive profiles are programmatically down-scaled using a capped Behavioral Multiplier, ensuring recruiters are only presented with active, responsive candidates.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**
 - **Experience Target:** A specified sweet spot of 6.0–8.0 years of experience, with acceptable bounds of 5.0–9.0 years.
 - **Technical Competency:** Hands-on production experience with embeddings, vector search indexes (Pinecone, FAISS, Milvus), Python, and ranking evaluation metrics (NDCG, MAP, MRR).
 - **Target Exceptions:** Explicit disqualification of candidates who have solely worked at major IT consulting/service companies (e.g., TCS, Infosys, Wipro, Cognizant) without product company exposure.

JD Understanding & Candidate Evaluation

- **Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?**
 - **Experience Curve:** I use a piecewise function that awards a score of 1.0 for the sweet spot, 0.8 for acceptable bounds, and drops to 0.1 for major outliers, preventing the selection of overqualified or underqualified candidates.
 - **Skill Verification:** Rather than checking for the existence of a word, I calculate a skill trust score utilizing:
$$\text{Proficiency Multiplier} \times \left(1 + \frac{\text{Duration}}{12}\right) \times \left(1 + \frac{\text{Endorsements}}{50}\right)$$
 - **Career Path Balance:** Candidates with a mix of service and product company histories are preserved, while pure-service candidates are programmatically filtered out.
 - **Behavioral Multipliers:** I apply a multiplicative modifier derived from live platform engagement (recruiter response rate, profile view metrics, and open-to-work status) to ensure reachability.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

- I implement a streaming reader that loads candidate JSON records line-by-line, scoring and compiling features instantly to keep execution time extremely fast.
- For each candidate, I generate a normalized profile score (comprising technical skills and experience alignment) and multiply it by a clamped platform-activity factor.

- **What models, algorithms, or heuristics are used?**

- **Piecewise Experience Curve:** A deterministic lookup heuristic to map years of experience directly to JD targets.
- **Logarithmic/Linear Skill Scaling:** A trust formula that evaluates proficiency levels (expert to beginner), duration months, and endorsement metrics.
- **Platform Activity Multipliers:** A clamping function restricting behavioral modifiers between a 0.5 penalty and a 1.3 boost.

Ranking Methodology

- **How are multiple candidate signals combined into a final ranking?**

- I calculate the composite score using a weighted linear combination:

$$\text{Profile Score} = (0.7 \times \text{Normalized Skill Score}) + (0.3 \times \text{Experience Score})$$

- The final rank score is then computed as:

$$\text{Final Score} = \text{Profile Score} \times \text{Behavioral Multiplier}$$

- All candidates are sorted in descending order of their final scores, using candidate_id alphabetically ascending as a strict tie-breaker.

Explainability & Data Validation

- **How are ranking decisions explained?**

I generate a structured, factual reasoning string directly from the candidate's raw profile fields. By compiling exact numbers (e.g., "Backend Engineer with 6.9 yrs experience; 2 matched AI skills"), the reasoning is transparent, clean, and directly verifiable.

- **How do you prevent hallucinations or unsupported justifications?**

Because my reasoning is constructed programmatically from the sanitized candidate record (rather than generated by an unsupervised LLM on-the-fly), hallucinations are mathematically impossible. Every claim in my reasoning string maps directly to a verified database field.

- **How does your solution handle inconsistent, low-quality, or suspicious profiles?**

I run defensive sanitization checks. If a candidate lists a required skill but has `duration_months = 0` or `endorsements = 0`, our trust-multipliers automatically scale their scoring down to negligible levels. Candidates with only consulting histories are hard-filtered out.

End-to-End Workflow

- **What is the complete workflow from JD input to ranked candidate output?**

I can map our workflow from start to finish as follows:

- **Step 1:** Initiation & JD Parsing

The system reads the fixed Job Description and initializes the target bounds (experience limits, required skills list, service company exclusions) inside `src/jd_matcher.py`.

- **Step 2:** File Streaming

`rank.py` opens `data/candidates.jsonl` and streams candidate records line-by-line to conserve memory.

- **Step 3:** Verification & Filtering

The `has_only_service_history` function checks the candidate's entire career history. If they have only worked at service companies, they are discarded.

- **Step 4:** Fact Extraction & Scoring

Skills are matched against the JD, and their trust scores are computed. Experience alignment is scored. Behavioral engagement multipliers are calculated from platform activity.

End-to-End Workflow

- **What is the complete workflow from JD input to ranked candidate output?**

- **Step 5:** Composite Scoring

The profile score is computed (70% weighted skills + 30% experience) and scaled by the behavioral multiplier to yield the final score and compile the reasoning text.

- **Step 6:** Sorting & Exporting

All scored records are sorted descending by score, with a deterministic tie-breaker on candidate ID (ascending).

The top 100 entries are sliced, assigned ranks 1 to 100, and exported to data/submission.csv.

System Architecture

- **Modular Python Design Pattern:**

- **src/jd_matcher.py:** Stores the target requirements (skills, experience, and service company exclusions) in an isolated, lowercase configuration class.
- **src/candidate_parser.py:** Contains the analytical functional blocks: has_only_service_history (exclusion logic), calculate_skill_score (trust formula calculation), and calculate_experience_score (experience alignment).
- **src/ranking_engine.py:** Core coordinator that normalizes raw scores, weights the profile elements (70/30), applies behavioral multipliers, and builds reasoning.
- **rank.py:** Streaming orchestrator that runs file I/O, executes sorting with deterministic tie-breaking, and outputs files.

System Architecture

- **Architectural Quality Attributes:**

- **$O(1)$ Memory Footprint:** By using file streaming instead of reading the entire uncompressed dataset into lists, memory utilization remains constant at under 20 MB throughout execution.
- **Deterministic Execution:** Eliminates random sorting behavior. Ties are broken using lexicographical candidate ID sorting.
- **Strict Data Sanitization:** Protects against honeypots by checking and discounting impossible skill durations and corrupted profiles.

Results & Performance

- **What results or insights demonstrate ranking quality?**
 - **Sweet-Spot Targeting:** Candidates in our Top 10 consistently match our sweet-spot criteria (e.g., CAND_0000031 with 6.0 years of experience, core verified AI skills, and a high response rate of 0.91).
 - **Defense Against Spammers:** Keyword-stuffed resumes containing target terms but zero months of use are scored at 0.00, pushing them far down the rank list.
 - **Honeypot Shield:** All impossible profiles are flagged and blocked from reaching the Top 100.
- **How does your solution meet the challenge's runtime and compute constraints?**
 - **Blazing Execution Speed:** Processes and scores all 100,000 deep profiles in 13.46 seconds on a standard CPU-utilizing just 4.4% of the 5-minute time limit.
 - **Memory Compliant:** Operates far below the 16 GB memory ceiling (typically under 20 MB).
 - **100% Validated:** The output was run through the official validate_submission.py script and confirmed as fully valid and format-compliant.

Technologies Used

- **What technologies, frameworks, and tools were used and why were they selected for this solution?**

- **Language: Python 3.12.3**

- Selected for its memory management optimizations and robust streaming I/O.

- **Built-in Standard Libraries Only (json, csv, time)**

- By engineering my entire system using Python's native libraries, I have zero external dependencies.
 - This makes my codebase exceptionally stable, immune to library version drifts, and incredibly easy for the judges to reproduce inside their sandbox environment instantly.

- **Version Control: Git + GitHub**

- Used for maintaining clean, logical save-points and documenting authentic development iterations.

Submission Assets

- **Submission Deliverables:**

- **Factual Ranked Output:** Exactly 100 candidate rows matching all rules, saved in *data/submission.csv* .
- **GitHub Repository:** Clean, commented code structure, complete with a structured *README.md* and *submission_metadata.yaml* .
- **Official Metadata Config:** Fully populated *submission_metadata.yaml* with hardware specs and reproduction commands.



INDIA.RUNS

Build what next India runs on

THANK YOU